esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

YallaFood

Plateforme de commande et de livraison de repas.

de repas auprès de différents restaurants



Sommaire

01	Description du sujet	06	Interactions entre les services
02	Problématique	07	Diagramme d’architecture globale
03	Acteurs et fonctionnalités	08	Planning et Risque
04	Découpage en microservices	09	Organisation Git
05	Données gérées par chaque service	10	Conclusion

L’équipe



Manel Hosni



Maryem Bdira Filali



Douaa Ben Rejab



Mahdi Benfattoum

01.Présentation du projet



YallaFood est une plateforme numérique qui facilite la relation entre les clients et les restaurants.



Elle permet aux clients de consulter les restaurants, découvrir leurs menus, commander des repas à emporter ou demander une livraison.

Elle offre également aux restaurateurs un espace dédié pour gérer leurs restaurants, menus et commandes.

02.problématique



Comment concevoir une plateforme numérique centralisée permettant de simplifier la gestion des restaurants, de faciliter la commande de repas et d'assurer un suivi efficace des livraisons ?



03 – Acteurs et fonctionnalités



1. Client

- Consulter les restaurants et leurs menus.
- Réserver une table.
- Commander des repas.
- Suivre ses commandes et livraisons.



2.Restaurateur

- Gérer les informations du restaurant.
- Ajouter et modifier les menus et les plats.
- Gérer les disponibilités.
- Traiter les réservations et les commandes.

03 – Acteurs et fonctionnalités



3 Administrateur

- Superviser les restaurants et les menus.
- Valider ou refuser les restaurants.
- Gérer les livreurs.
- Superviser la plateforme.



4. Livreur

- Consulter les livraisons assignées.
- Mettre à jour le statut des livraisons.
- Envoyer sa position.
- Gérer sa disponibilité.

04.Découpage en microservices



Architecture **Microservices**

Utilisateur

Comptes, profils et authentification

Restaurant

Informations des restaurants

Menu

Menus, catégories, plats et ingrédients

Commande

Commandes et détails

Livraison

Livreurs et suivi des livraisons

Microservice commun Utilisateur



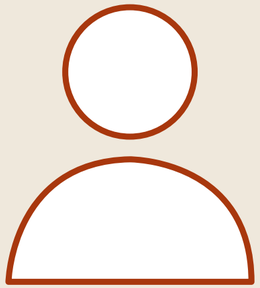
Le microservice Utilisateur est chargé de gérer les comptes et l’accès à la plateforme YallaFood.

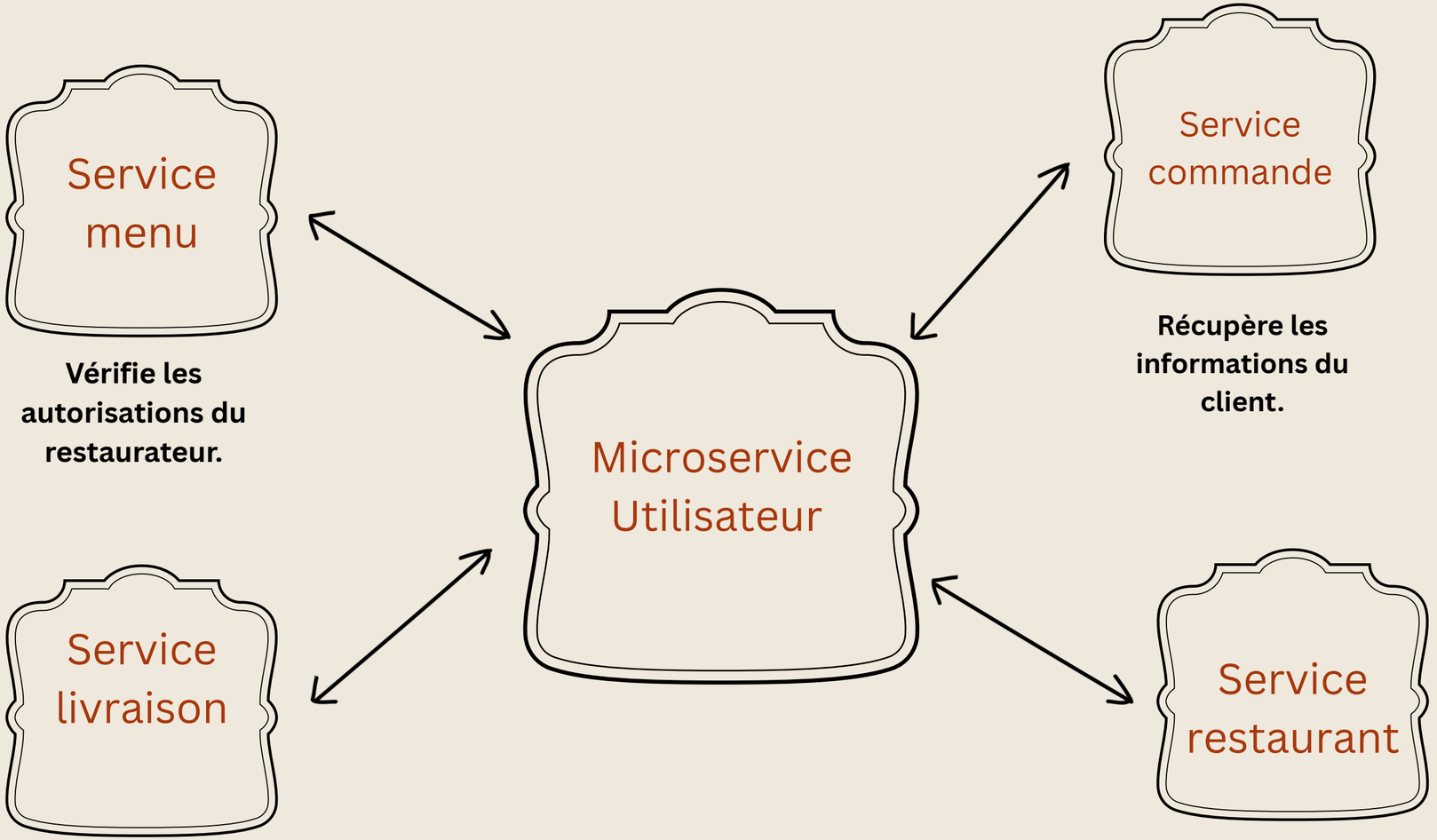
Il permet aux utilisateurs :
de créer un compte, de se connecter et de gérer leur profil.

Il permet également à la plateforme :
d’identifier chaque utilisateur et de contrôler ses droits d’accès selon son rôle (client, restaurateur, livreur, administrateur).

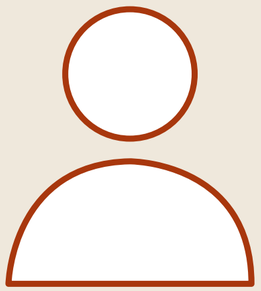
Interaction avec les autres microservices:







Interaction et types de communication



Synchrone · REST / HTTP

Service Restaurant
Vérifie l'identité et le rôle du restaurateur.

Service Menu
Vérifie les autorisations avant la gestion des menus.

Service Commande
Récupère l'identité du client pour associer ses commandes.

Service Livraison
Récupère l'adresse du client pour la livraison.



Service Utilisateur
Gestion des restaurants et de leurs disponibilités

Asynchrone · RabbitMQ

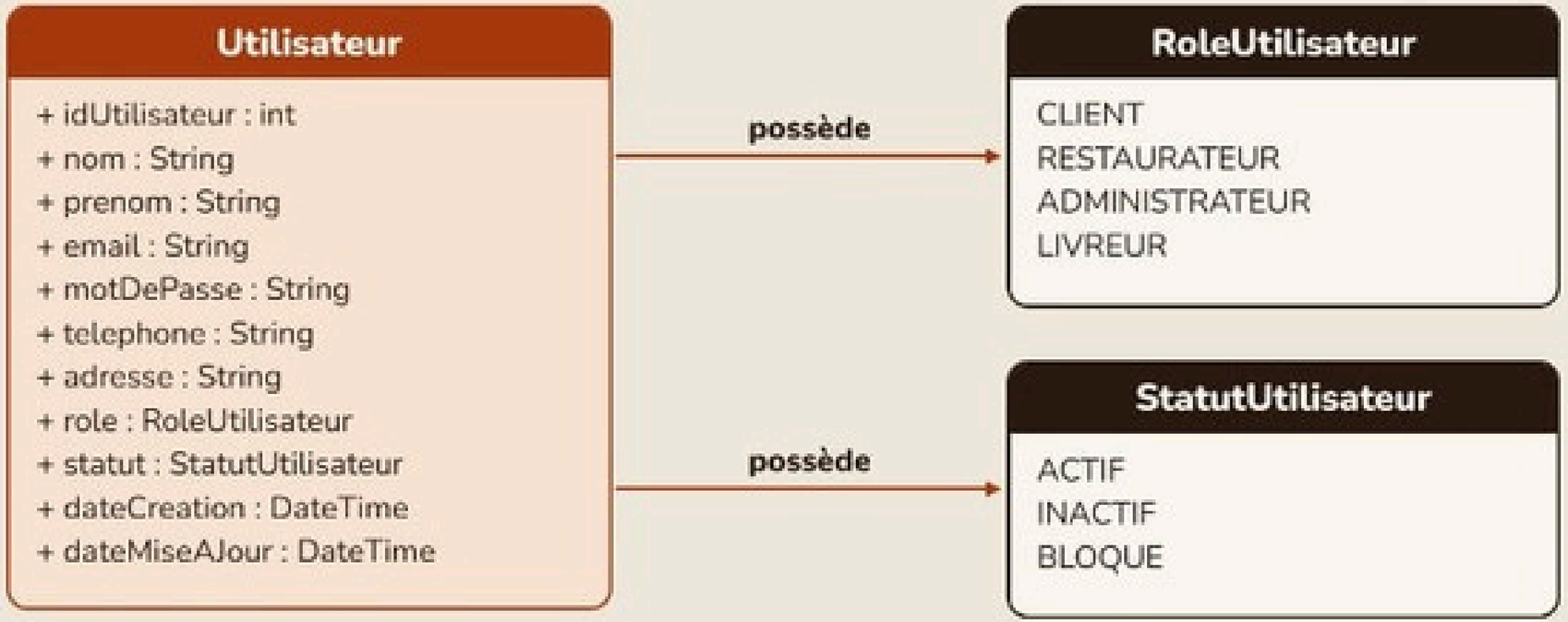
✓ Événements utilisateur :

- Création d'un compte
- Modification du profil
- Désactivation d'un compte
- Mise à jour des informations utilisateur

⟷ Communication synchrone (REST / HTTP)

⇢ Communication asynchrone (RabbitMQ)

Diagramme de classes du microservice Utilisateur



Données : table MySQL « utilisateur », adresse e-mail unique, mot de passe stocké de façon sécurisée. Les autres services référencent l'identifiant de l'utilisateur.

Microservice Restaurant





Le microservice Restaurant est chargé de gérer les établissements présents sur la plateforme YallaFood.

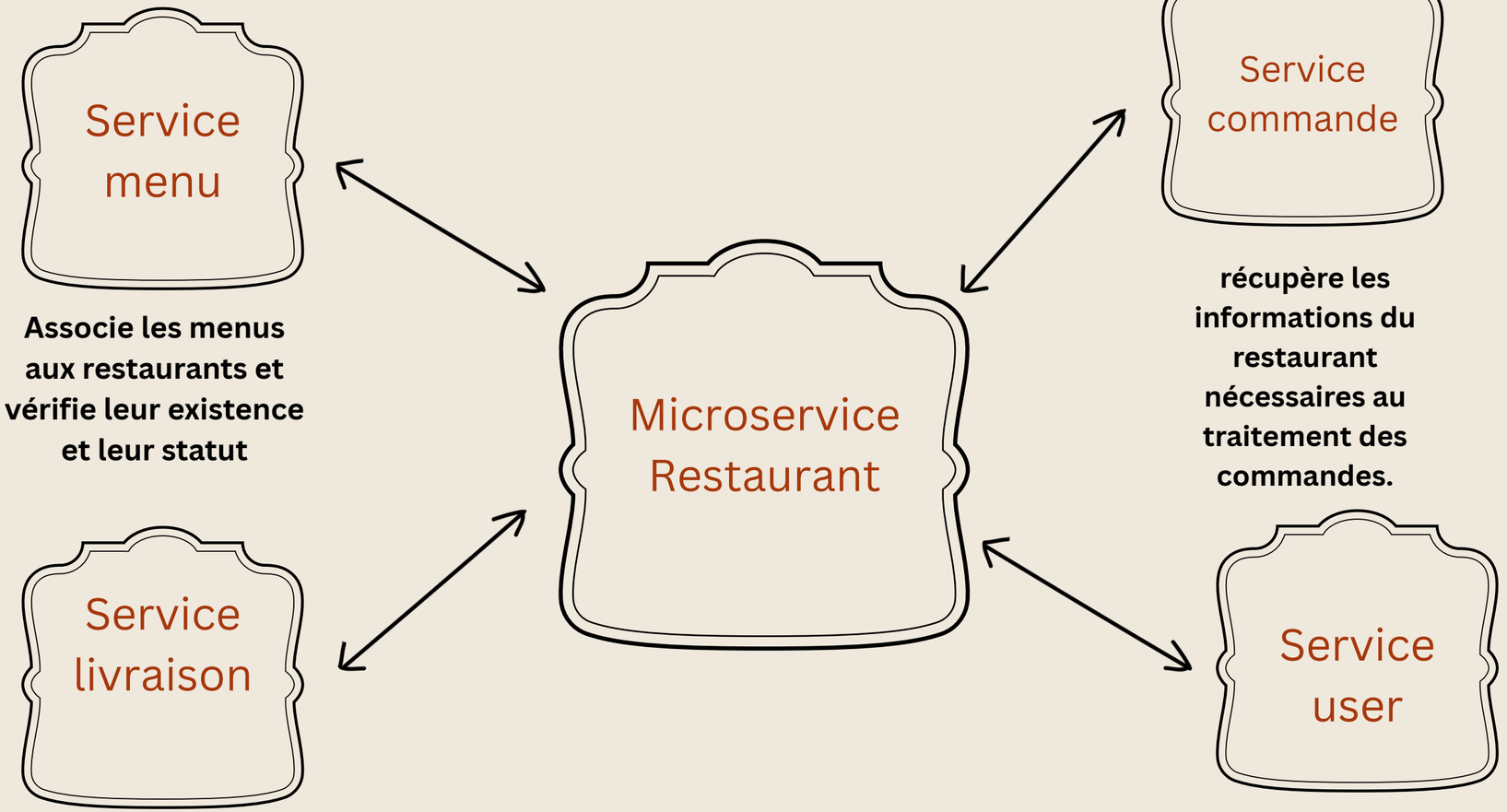
Il permet aux restaurateurs :
d'ajouter,modifier et gérer les informations de leurs restaurants

Il permet également aux clients:
de rechercher des restaurants et de consulter leurs informations avant de réserver ou de commander.

Interaction avec les autres microservices:







Associe les menus aux restaurants et vérifie leur existence et leur statut

récupère les informations du restaurant nécessaires au traitement des commandes.

récupère l'adresse du restaurant pour organiser le retrait des commandes.

Vérifie l'identité et les autorisations des restaurateurs.

Interaction et types de communication



Synchrone · REST / HTTP

Service Utilisateur
Vérifie l'identité des restaurateurs

Service Menu
Associe les menus aux restaurants

Service Commande
Récupère les infos du restaurant

Service Livraison
Récupère l'adresse du restaurant

Service Restaurant
Gestion des restaurants et de leurs disponibilités

Asynchrone · RabbitMQ

Autres services

- Désactivation d'un restaurant
- Modification des informations
- Mise à jour des disponibilités
- Notification de validation ou de refus

Communication synchrone (REST / HTTP)

Communication asynchrone (RabbitMQ)

Endpoints REST du Service Restaurant

Synchrone · REST / HTTP

Endpoint	Rôle	Codes de réponse
GET /restaurants	Récupérer la liste des restaurants	200 OK
GET /restaurants/{id}	Détails d'un restaurant	200 OK / 404 Not Found
POST /restaurants	Créer un restaurant	201 Created
PUT /restaurants/{id}	Mettre à jour un restaurant	200 OK / 404 Not Found
DELETE /restaurants/{id}	Supprimer un restaurant	204 No Content
GET /restaurants/search	Rechercher un restaurant (par ville, cuisine…)	200 OK

Service Restaurant Gestion des restaurants et de leurs disponibilités

Événements et exemples de réponses

Asynchrone · RabbitMQ

Événement	Description
restaurant.updated	Informations d'un restaurant modifiées
restaurant.disabled	Un restaurant est désactivé
restaurant.availability.updated	Disponibilités modifiées (horaires, tables)
restaurant.validation	Notification de validation ou de refus

Exemples de réponses

200 OK
Succès d'une requête

ex : récupération d'un restaurant

404 Not Found
Ressource introuvable

ex : restaurant inexistant

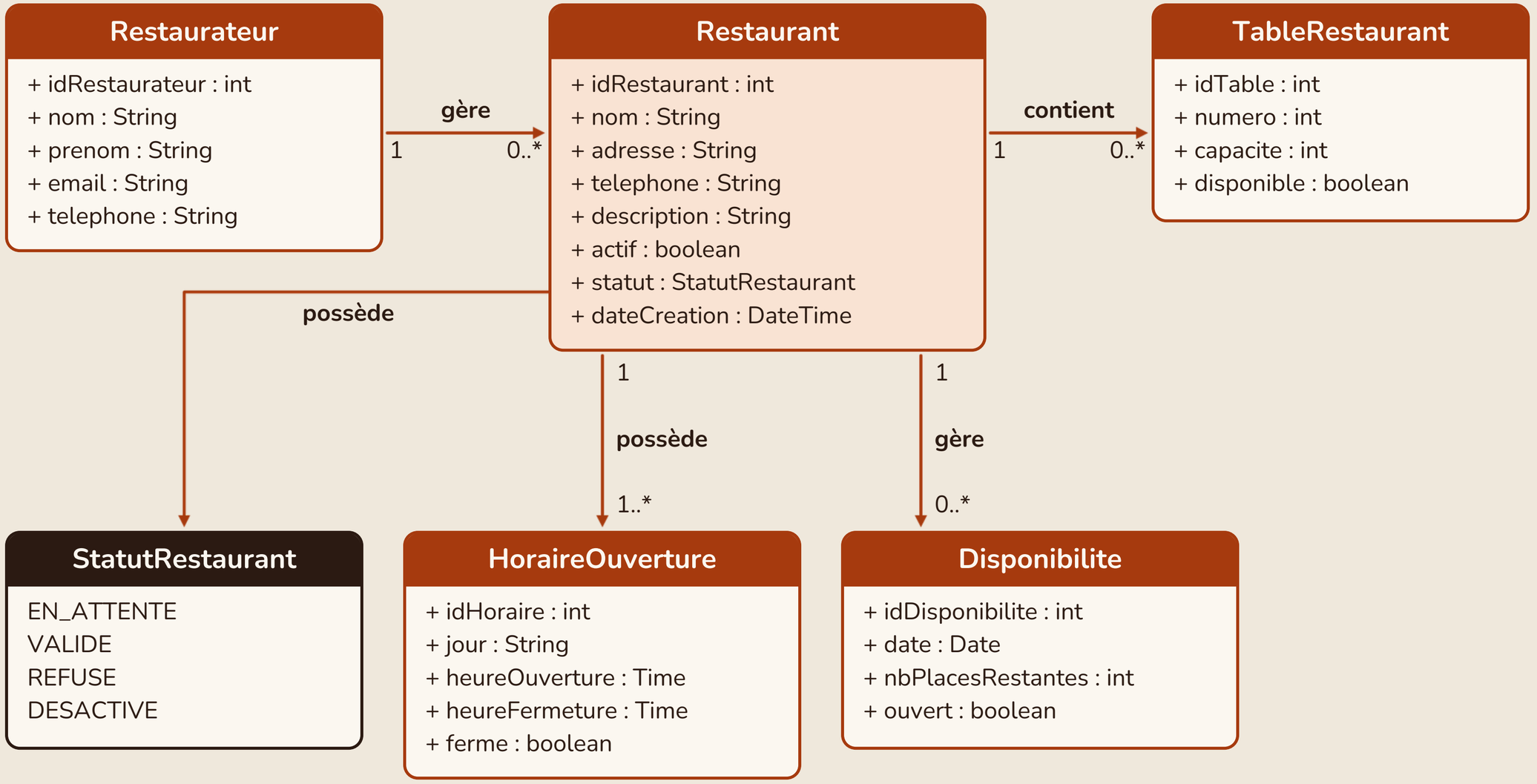
400 Bad Request
Requête invalide

ex : données manquantes

⟷ Communication synchrone (REST / HTTP) ----► Communication asynchrone (RabbitMQ)

Diagramme de classes du Service Restaurant





Microservice Menu

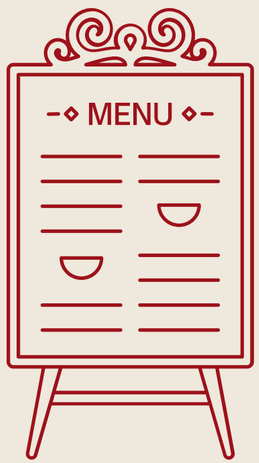
Le microservice Menu assure la gestion des menus et des plats des restaurants, permettant aux restaurateurs d'organiser leur offre et aux clients de découvrir les plats disponibles.

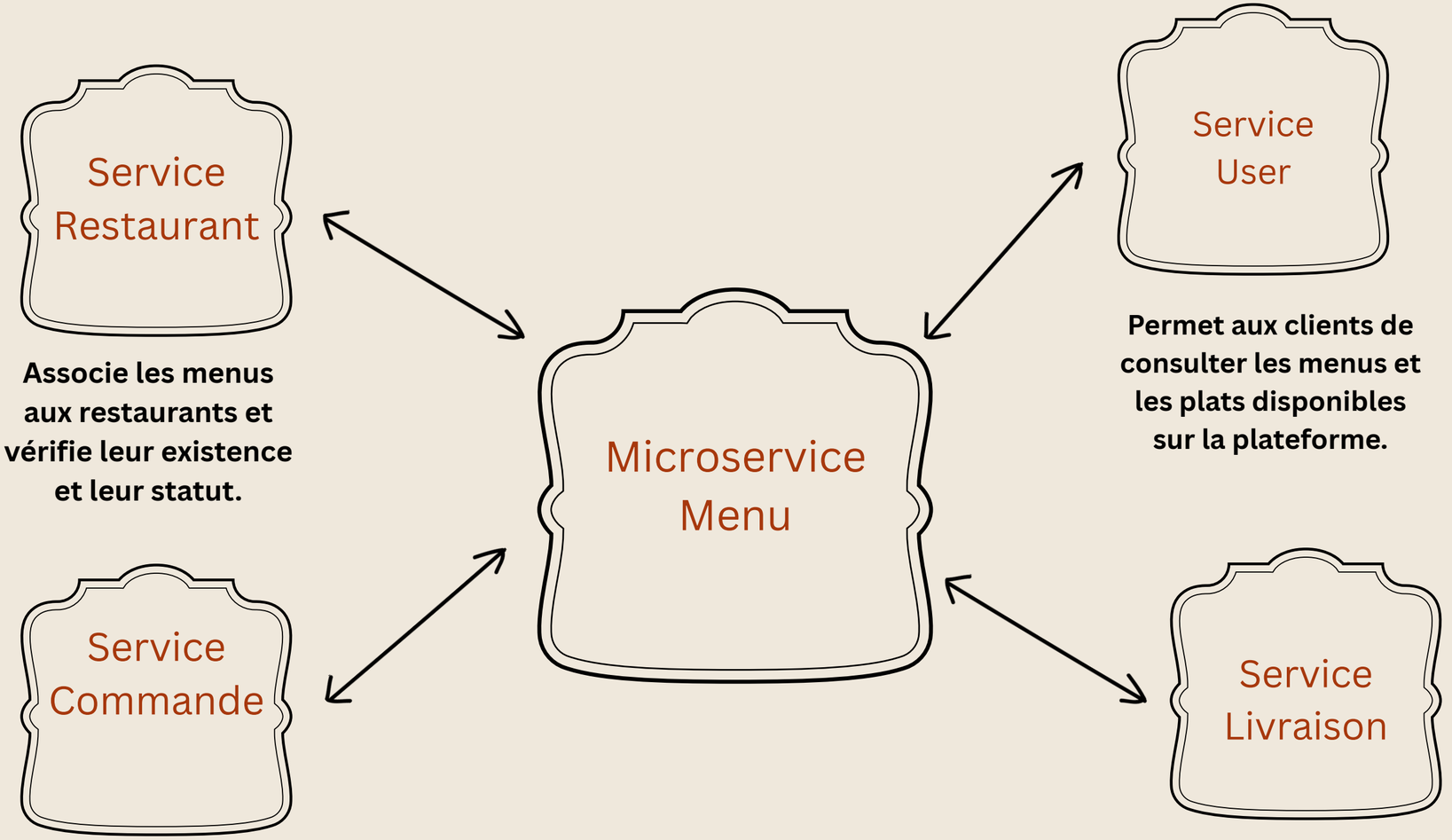


Ses principales responsabilités :

- Gestion des menus des restaurants.
- Organisation des plats par catégories.
- Gestion des informations des plats : nom, description, prix et image.
- Gestion des ingrédients associés aux plats.
- Consultation des menus et des plats.

Interaction avec les autres microservices:





Associe les menus aux restaurants et vérifie leur existence et leur statut.

Permet aux clients de consulter les menus et les plats disponibles sur la plateforme.

Récupère les informations des menus et des plats (nom, prix et disponibilité) nécessaires à la création des commandes.

Utilise indirectement les informations des plats à travers le service Commande pour assurer le traitement des livraisons.

Interaction et types de communication



Synchrone · REST / HTTP

Service Restaurant
Vérifie l'existence et l'état du restaurant

Client (via API Gateway)
Consulte le menu actif d'un restaurant (catégories, plats)

Service Livraison
Récupère le nom, le prix et la disponibilité d'un plat

Service Menu

Gestion des menus, catégories et plats

Asynchrone · RabbitMQ

Autres services

- ✓ Publication d'un menu (MenuPublished)
- ✓ Archivage d'un menu (MenuArchived)
- ✓ Modification du prix ou suppression d'un plat
- ✓ Mise à jour de la disponibilité d'un plat

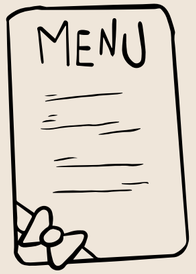
Communication synchrone (REST / HTTP)

Communication asynchrone (RabbitMQ)

Endpoints REST du Service Menu : Menus

Synchrone · REST / HTTP

Endpoint	Rôle	Codes de réponse
POST /api/menus	Créer un menu	201 Created / 400 / 404 Restaurant introuvable / 409 Nom déjà utilisé
GET /api/menus?restaurantId=&statut=	Lister les menus (filtres)	200 OK
GET /api/menus/{id}	Détails d'un menu	200 OK / 404 Not Found
GET /api/menus/{id}/carte	Carte complète (catégories et plats)	200 OK / 404
PUT /api/menus/{id}	Modifier un menu	200 OK / 400 / 404
PATCH /api/menus/{id}/statut	Activer / désactiver un menu	200 OK / 404
DELETE /api/menus/{id}	Désactiver un menu (suppression logique)	204 No Content / 404



Service Menu Gestion des menus,

Événements et exemples de réponses

Asynchrone · RabbitMQ

Événement	Description
MenuPublished	Un menu est publié et visible par les clients
MenuArchived	Un menu est archivé et n’est plus proposé aux clients
PlatAvailabilityUpdated	Disponibilité d’un plat modifiée
PlatPriceUpdated	Prix d’un plat modifié
PlatDeleted	Un plat est supprimé d’une catégorie



Exemples de réponses

200 OK

Succès d’une requête

ex : consultation d’un menu

404 Not Found

Ressource introuvable

ex : restaurant ou menu inexistant

400 Bad Request

Requête invalide

ex : données manquantes

Communication synchrone (REST / HTTP)

Communication asynchrone (RabbitMQ)

Diagramme de classes du Service Menu



Menu
+ idMenu : int
+ idRestaurant : int
+ nom : String
+ description : String
+ actif : boolean
+ statut : StatutMenu
+ dateCreation : DateTime

contient
1 — 0..*

Categorie
+ idCategorie : int
+ nom : String
+ description : String
+ ordre : int

regroupe
1 — 0..*

Plat
+ idPlat : int
+ nom : String
+ description : String
+ prix : double
+ imageUrl : String
+ disponible : boolean
+ tempsPreparation : int

possède

StatutMenu
BROUILLON
PUBLIE
ARCHIVE

0..*
composé de
0..*

Ingredient
+ idIngredient : int
+ nom : String
+ allergene : boolean

Microservice Commande





Le microservice Commande est responsable de la gestion des commandes de repas des clients sur la plateforme YallaFood.

Ses principales fonctionnalités :

- Créer une nouvelle commande.
- Consulter les commandes et leurs détails.
- Suivre le statut d'une commande.
- Annuler une commande sous certaines conditions.
- Confirmer ou refuser une commande.
- Gérer les commandes à emporter et en livraison.

Interaction avec les autres microservices:





Service Restaurant

Vérifier que le restaurant est actif

Service utlisateur

Identifier le client

Microservice Commande

Service livraison

Créer et suivre la livraison

Service Menu

Vérifier le prix et la disponibilité des plats

Interaction et types de communication





Synchrone · REST / HTTP

Service Utilisateur
- Identifier le client.
- Vérifier les informations du client.

Service Menu
- Vérifier le prix des plats.
- Vérifier la disponibilité des plats.

Service Restaurant
- Vérifier l'existence du restaurant.
- Vérifier que le restaurant est actif.

Service Livraison
- Transmettre les informations de la commande confirmée.
- Demander la création d'une livraison.



Service Commande
- Créer et gérer les commandes.
- Enregistrer les commandes dans MySQL.
- Gérer les statuts des commandes.

Asynchrone · RabbitMQ

Autres services
- ✓ Nouvelle commande créée
- ✓ Commande confirmée
- ✓ Commande refusée
- ✓ Commande prête
- ✓ Commande annulée

⟷ Communication synchrone (REST / HTTP)

⇢ Communication asynchrone (RabbitMQ)

Endpoints REST du Service Commande : Commande

Synchrone · REST / HTTP

Endpoint	Rôle	Codes de réponse
POST /api/commandes	Créer une commande	201 Created / 400 / 404 Restaurant introuvable / 409 Commande déjà fait
GET/api/commandes	Afficher les commandes	200 OK
GET /api/commandes/{id}	Consulter une commande	200 OK / 404 Not Found
PATCH /api/commandes/{id}/statut	Modifier le statut	200 OK / 404
DELETE /api/commandes/{id}	Annuler une commande	204 No Content / 404

Service Commande Gestion des commandes

Événements et exemples de réponses

Asynchrone · RabbitMQ



Événement	Description
CommandeCreeeEvent	nouvelle commande créée.
CommandeConfirmeeEvent	commande confirmée.
CommandeRefuseeEvent	commande refusée.
CommandePreteEvent	commande prête.
CommandeAnnuleeEvent	commande annulée.

Exemples de réponses

✓ 200 OK

Succès d'une requête

ex : Creation d'une commande

404 Not Found

Ressource introuvable

ex : restaurant inexistant

⚠ 400 Bad Request

Requête invalide

ex : données manquantes

Communication synchrone (REST / HTTP)

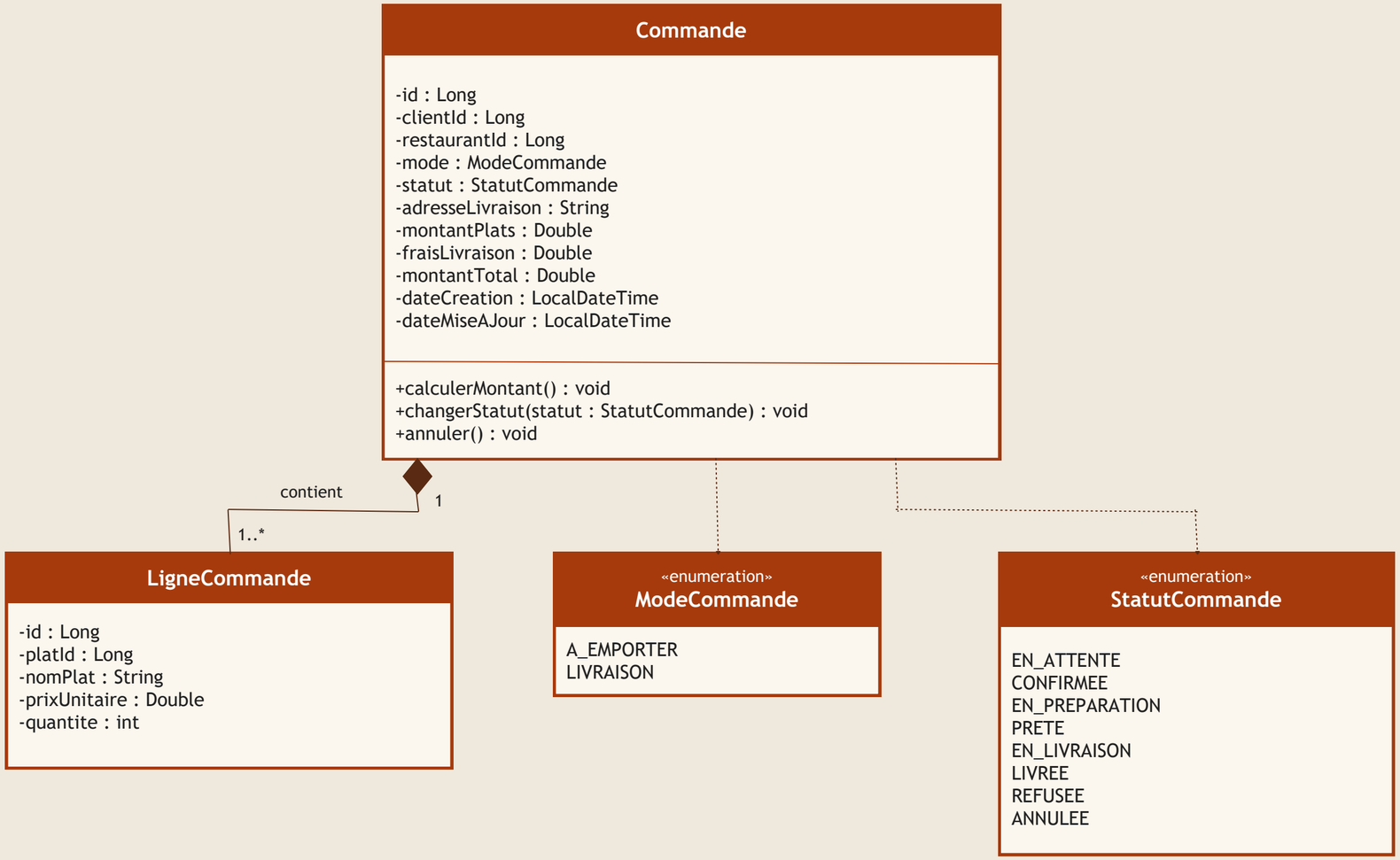
Communication asynchrone (RabbitMQ)

Diagramme de classes

Structure du microservice Commande







Microservice Livrasion





Le microservice Livraison est responsable de la gestion des livraisons des commandes de repas sur la plateforme YallaFood.

Ses principales fonctionnalités :

- Créer une livraison à partir d’une commande confirmée.
- Assigner un livreur disponible.
- Suivre le statut et la position d’une livraison.
- Mettre à jour le statut (récupérée, en route, livrée, échouée).
- Annuler une livraison avant la récupération du repas.
- Gérer les livreurs et leur disponibilité.

Interaction avec les autres microservices:





Service Restaurant

Récupérer l'adresse de retrait

Microservice Livraison

Service Notification

Prévenir le client

Service Commande

Recevoir la demande de livraison et informer du statut

Interaction et types de communication





Synchrone · REST / HTTP

Asynchrone · RabbitMQ

Service Commande

- demander la création de la livraison
- Vérifier la disponibilité des plats.

Service Restaurant

- récupérer l'adresse de retrait
- vérifier que le restaurant existe.



Service Livraison

- **Créer et gérer les livraisons.**
- **Assigner un livreur disponible.**
- **Enregistrer les livraisons dans MySQL.**
- **Gérer les statuts de livraison.**

Autres services

- ✓ livraison assignée
- ✓ livraison en cours
- ✓ livraison terminée
- ✓ livraison échouée
- ✓ livraison annulée.

⟷ Communication synchrone (REST / HTTP)

⇢ Communication asynchrone (RabbitMQ)

Endpoints REST du Service Livraison : Livraison

Synchrone · REST / HTTP

Endpoint	Rôle	Codes de réponse
POST /api/livraisons	Créer une livraison	201 Created / 400 / 404 Commande introuvable / 409 Livraison déjà créée
GET /api/livraisons/{id}	Consulter une livraison	200 OK / 404 Not Found
GET /api/livraisons/{id}/suivi	Consulter le suivi et la position	200 OK / 404 Not Found
PATCH /api/livraisons/{id}/statut	Modifier le statut	200 OK / 404/409 Transition interdite
PUT /api/livraisons/{id}/livreur	Assigner un livreur	200 OK / 404 / 409 Livreur indisponible
DELETE /api/livraisons/{id}	Annuler une livraison	204 No Content / 404 / 409

Service Livraison Gestion des Livraison

Endpoints REST du Service Livraison : Livraison

Synchrone · REST / HTTP

Endpoint	Rôle	Codes de réponse
POST /api/livreurs	Ajouter un livreur	201 Created / 400
GET /api/livreurs?disponible=true	Afficher les livreurs disponibles	200 OK
GET /api/livreurs/{id}	Consulter un livreur	200 OK / 404 Not Found
PUT /api/livreurs/{id}	Modifier un livreur	200 OK / 400 / 404
PATCH /api/livreurs/{id}/disponibilite	Changer la disponibilité	200 OK / 404
DELETE /api/livreurs/{id}	effacer un livreur	204 No Content / 404 / 409

Service Livraison Gestion des Livraison

Événements et exemples de réponses

Asynchrone · RabbitMQ



Événement	Description
LivraisonAssigneeEvent	livreur assigné à la livraison.
LivraisonEnCoursEvent	livraison en cours, livreur en route.
LivraisonTermineeEvent	commande livrée au client.
LivraisonEchoueeEvent	livraison échouée (client refuse ou injoignable).
LivraisonAnnuleeEvent	livraison annulée.

Exemples de réponses

201 Created

Succès d'une requête

ex : Creation d'une livraison

404 Not Found

Ressource introuvable

ex : livraison inexistant

409 Conflict

action impossible

ex: ivreur indisponible.

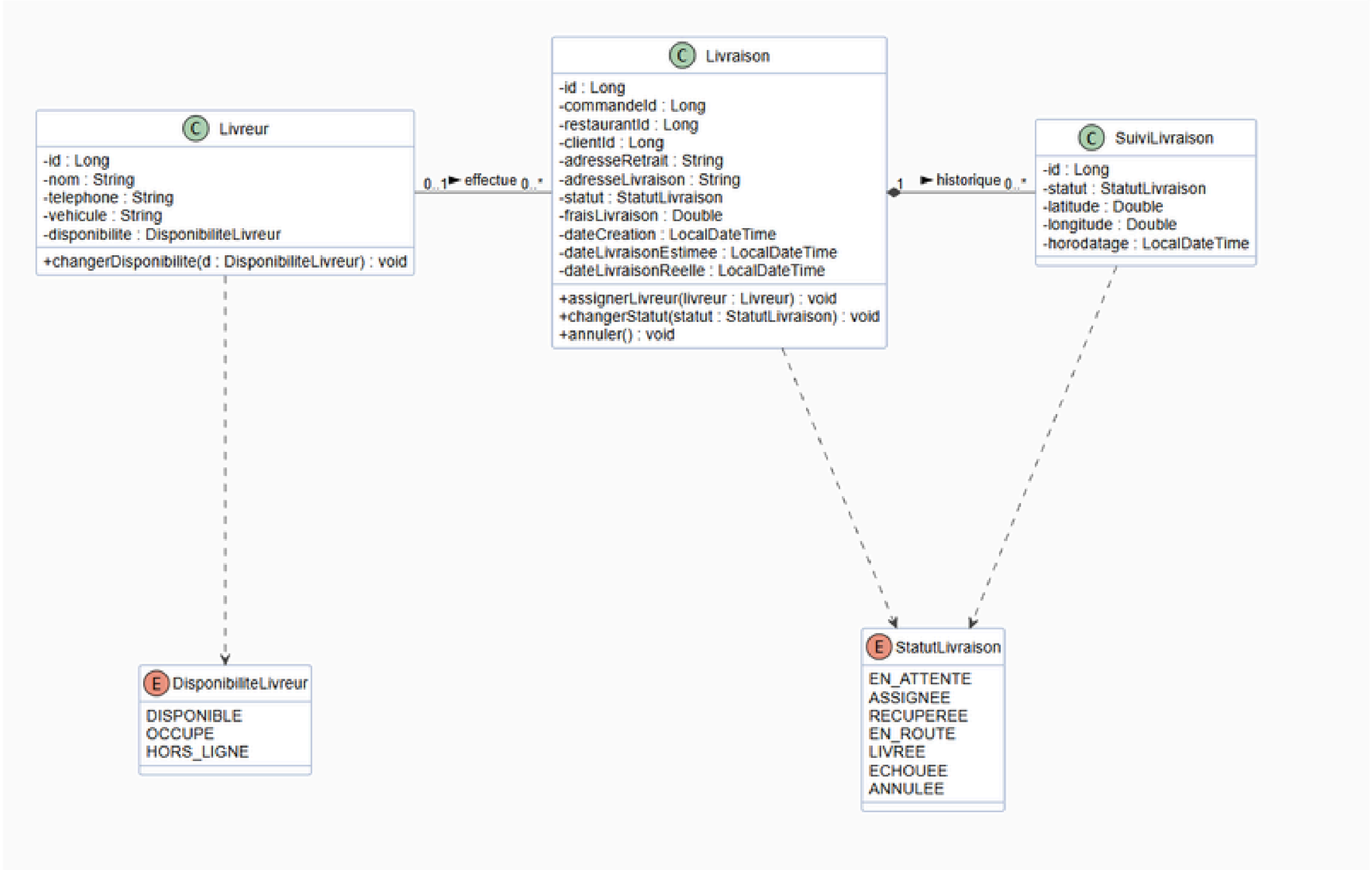
Communication synchrone (REST / HTTP)

Communication asynchrone (RabbitMQ)

Diagramme de classes



Structure du microservice Livrasion



05 –Données gérées par chaque service

Microservice	Base de données	Données principales gérées	Responsable
Livraison	MySQL (livraison_db)	Livraison, Livreur, Suivi_livraison	Mahdi Ben Fattoum
Restaurant	*MYsql (restaurant_db)*	*Restaurateur, Restaurant,TableRestaurant,Disponibilité,HoraireOuverture*	*Maryem Bdira Filali*
Menu	*MySQL (menu_db)*	*Menu, Catégorie, Plat, Ingrédient, Plat_Ingredient*	Manel Hosni
Commande	*MySql(commande_db)*	*Commande, Ligne_Commande, ModeCommande, StatutCommande*	*Douaa Ben Rejab*
User	*MySQL (utilisateur_db)*	*Utilisateur, RoleUtilisateur, StatutUtilisateur*	***tous les membres de groupes***

Chaque microservice possède sa propre base de données

05 –Données gérées par chaque service

Livraison : données gérées

Livraison

Références de la commande, du restaurant et du client · adresses · statut · frais · dates

Livreur

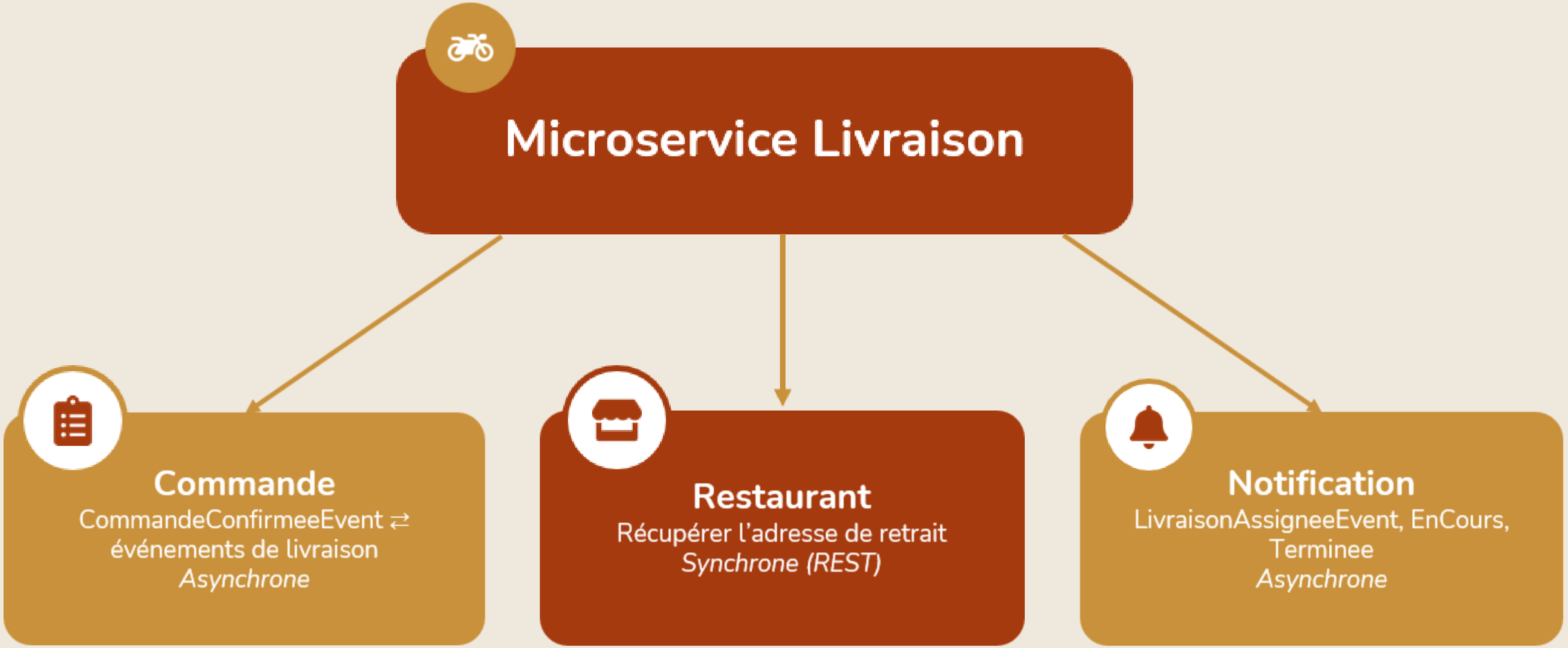
Nom · téléphone · véhicule · disponibilité (DISPONIBLE, OCCUPE, HORS_LIGNE)

Suivi_livraison

Historique des statuts et des positions GPS de chaque livraison

MySQL · base livraison_db · 3 tables

06 –Interactions entre les services



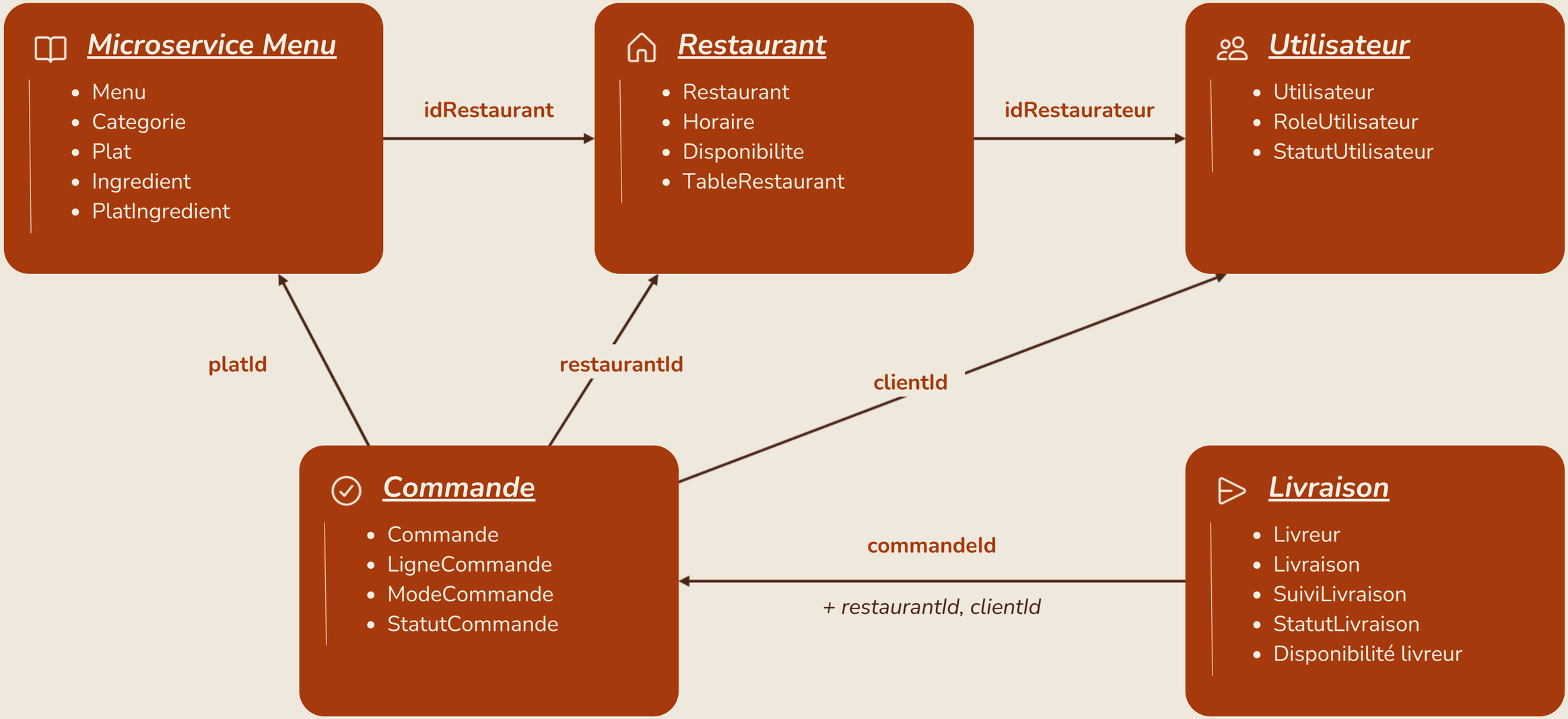
Synchrone = appel REST · Asynchrone = événement via le broker

06 –Interactions entre les services

Pourquoi communiquer ?

Interaction	Pourquoi ?	Données échangées	Type
Commande → Livraison	Déclencher la création de la livraison	CommandeConfirmeeEvent	Asynchrone
Livraison → Commande	Mettre à jour le statut de la commande	LivraisonEnCours, Terminee, Echouee	Asynchrone
Livraison → Restaurant	Connaître l'adresse de retrait	restaurantId, adresse	Synchrone
Livraison → Notification	Prévenir le client	livraisonId, statut	Asynchrone

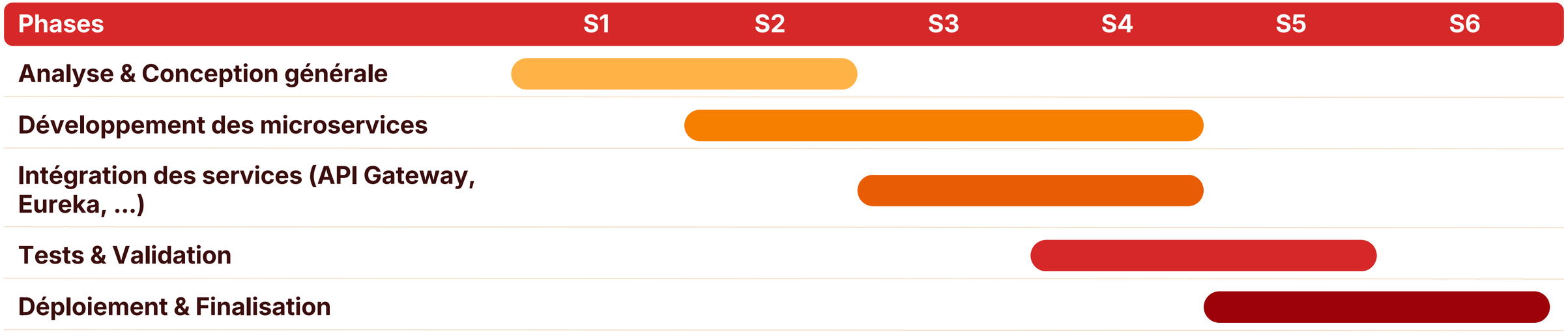
07 – Architecture globale des microservices



Une flèche = référence par identifiant vers un autre microservice.

Objectif : livrer une plateforme stable, sécurisée et performante dans un délai de 6 semaines.

Planning du projet · 6 semaines



Répartition des tâches

Service Utilisateur
Gestion des comptes, authentification

Service Restaurant
Gestion des restaurants et menus

Service Commande
Traitement des commandes

Service Livraison
Suivi et livraison des commandes

Service Menu
Suivi et gestion des menus

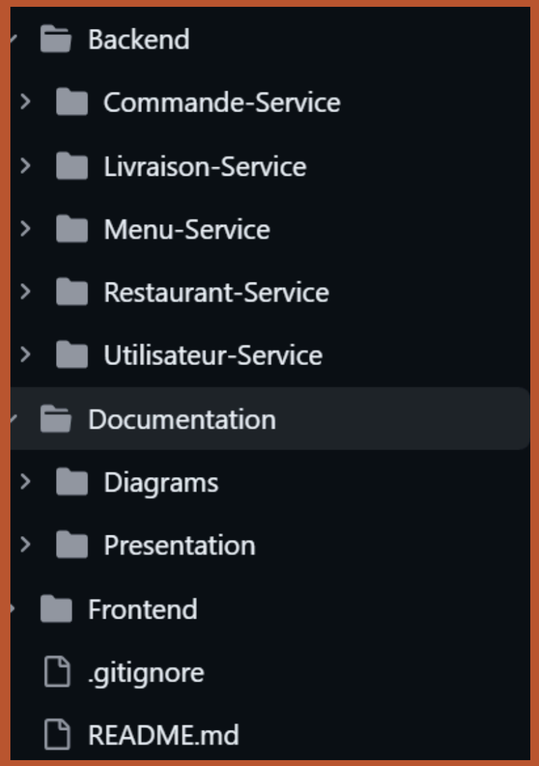
Communication : API REST + RabbitMQ

Risques identifiés et mesures préventives

Risque identifié	Description	Probabilité	Impact	Mesures préventives
Problèmes d'intégration	Difficultés à faire communiquer les microservices	Élevée	Élevé	Tests réguliers des API, bonne configuration d'Eureka et du Gateway
Retard de développement	Dépassement du délai prévu (6 semaines)	Moyenne	Élevé	Planning réaliste, suivi hebdomadaire
Panne d'un service	Indisponibilité d'un microservice	Moyenne	Élevé	Redondance, monitoring et tests de charge
Risques de sécurité	Accès non autorisé, fuite de données	Moyenne	Très élevé	Authentification JWT, contrôle des accès
Perte de données	Suppression ou corruption des données	Faible	Élevé	Sauvegardes régulières, validation des données
Dépendances externes	Problèmes avec des services tiers (RabbitMQ, ...)	Faible	Moyen	Plans de secours, alternatives possibles

Organisation du dépôt Git et outil de gestion du projet

Structure du dépôt



GitHub Projects : tableau Kanban

Tâches suivies en issues, par colonnes : À faire, En cours, En revue, Terminé.

Branches et pull requests

main (stable), develop et feature/service-tâche ; fusion après revue d'un coéquipier.

Un responsable par microservice

Chacun gère son service dans Backend/ ; le service commun est fait par toute l'équipe.

Enseignant ajouté comme collaborateur du dépôt

Conclusion

YallaFood : une seule plateforme pour découvrir, commander et se faire livrer.

Une plateforme centralisée

Clients, restaurateurs, livreurs et administrateur utilisent une seule application.

Une architecture modulaire

Cinq microservices indépendants, chacun avec sa base MySQL, reliés par API REST et RabbitMQ.

Prochaines étapes

Développer les services et le frontend, tester l’intégration, finaliser le dossier technique.



merci pour votre attention

